

UNIDADE 4

CRUD. Persistencia de datos e consumo de API REST en Vue

Obxectivos da unidade

- **Consumir unha API REST** dende *Vue* empregando **Axios** para realizar peticións HTTP (GET, POST, PUT e DELETE).
- Desenvolver un **CRUD** completo no *frontend*, xestionando datos de forma estruturada.
- Comprender a **persistencia de datos**, diferenciando entre datos en memoria e datos gardados nun backend (json-server).
- **Xestionar datos compartidos** entre compoñentes mediante un estado centralizado, mellorando a organización da aplicación.

A.1. Introducción

Na UD4 traballaremos operacións **CRUD** reais (crear, ler, actualizar e eliminar datos) dende a aplicación **Vue**. Para iso necesitamos unha **API backend** á que facer peticións **HTTP**.

A idea é crear unha aplicación para gardar os datos de usuarios e as tarefas asociadas a cada un deles.

En **aplicacións reais**, esta API adoita estar conectada a bases de datos como *MySQL*, *PostgreSQL*, *MongoDB*, etc, pero a configuración dunha delas para traballar con **VUE** suporía ir máis alá do que pretendemos.

Porén, neste curso non imos centrarnos no *backend*, senón en aprender ben:

- Como consumir unha *API* desde *Vue*.
- Como estruturar un *CRUD* no *frontend*.
- Como **xestionar datos compartidos** con estado centralizado.
- Como traballar con peticións *HTTP* reais.

Para acadar estes obxectivos imos empregar **json-server**, ferramenta que nos permite:

- Simular unha *API REST* completa.
- Traballar con datos persistentes.
- Usar **rutas reais** (*GET*, *POST*, *PUT*, *DELETE*)..
- Ver **respostas HTTP reais** (200, 404, 500...)

E todo elo sen ter que montar un *backend* complexo.

A clave é esta, e convén insistila varias veces:

- As librarías e a forma de traballar **son as mesmas**.
- **Axios** funciona igual con *json-server* que con unha *API* real.

Por exemplo, usar:

- `axios.get('/usuarios')`
- `axios.post('/tarefas', novaTarefa)`
- `axios.put('/usuarios/3', datos)`
- `axios.delete('/tarefas/5')`

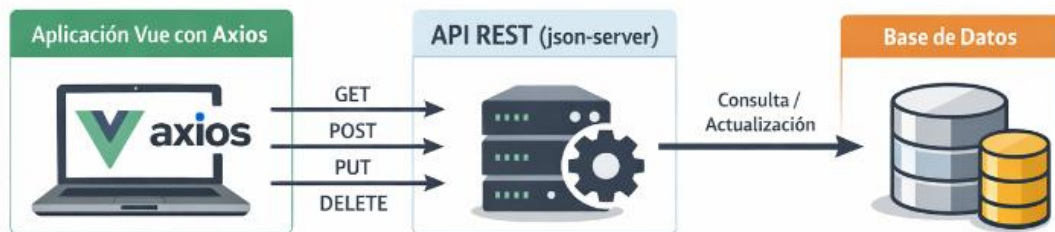
Non cambia absolutamente nada se detrás hai:

- *json-server*
- unha *API* en *Node + Express*
- unha *API* en *Java + Spring*
- unha *API* conectada a *MySQL*

O **frontend Vue** non sabe “nin lle importa” que tipo de base de datos hai detrás, de iso encárgase o *backend*; *Vue* e *Axios* só consumen datos unha API HTTP.

Ademais, cabe sinalar que usaremos como ferramenta **Axios** porque é unha das librerías máis empregadas no ecosistema **Vue** para facer peticións *HTTP*, sinxela de usar, moi clara a nivel didáctico e exactamente a mesma que se emprega en proxectos profesionais.

Axios actúa como unha **ponte entre a aplicación Vue e o backend**, permitindo comunicarse con calquera *API REST* independentemente da tecnoloxía que haxa detrás.



Para finalizar, no **almacén** da Unidade 4 temos os compoñentes *XestiónTarefas.vue* e *XestiónUsuarios.vue* con modificacións con respecto aos anteriores para poder traballar ao longo da unidade de forma cómoda as operacións CRUD. A idea é dar de alta, baixa, modificar e listar usuarios, así como as tarefas asociadas a cada un en base a un estado centralizado.

Se queredes conservar o anterior, convén crear unha nova rama se estades usando **GIT** como repositorio do que estamos a facer.

E xá para entrar en materia e ir adiantando instalamos *axios*:

```

round 0 vulnerabilities
\CURSORVUE\proxectovue> npm install axios

added 11 packages, and audited 234 packages in 2s

45 packages are looking for funding
  run `npm fund` for details
  
```

Recordamos que ao final desta unidade teremos unha pequena aplicación capaz de xestionar usuarios e tarefas consumindo unha API REST real, moi similar ao funcionamento de moitas aplicacións profesionais.

A.2. Configuración do backend simulado con json-server

json-server é unha ferramenta lixeira que nos permite crear unha *API REST*, ficticia pero realista, a partir dun simple ficheiro *JSON*.

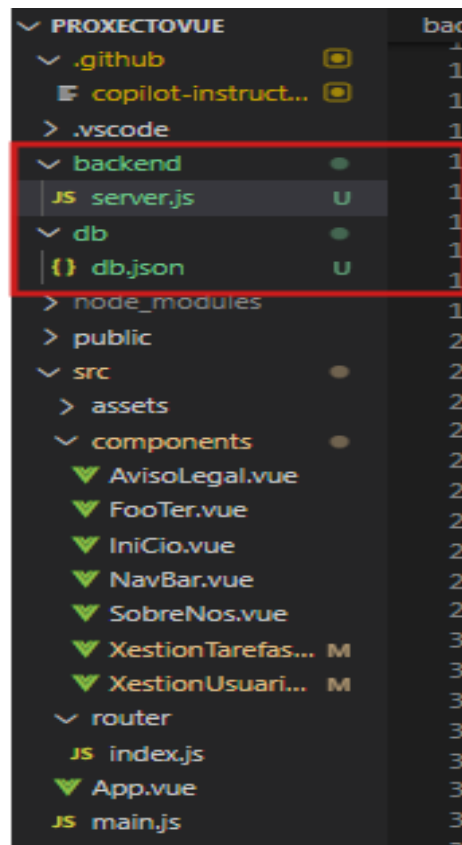
Sen configurar servidores reais nin bases de datos complexas, podemos simular un *backend* completo con operacións **CRUD** (*GET, POST, PUT, PATCH, DELETE*), ideal para aprender, probar e desenvolver aplicacións *frontend*.

En primeiro lugar imos crear o directorio **backend** na raíz do proxecto e, dentro del:

- **server.js**, que simulará un *backend* sinxelo con *json-server*

Noutra carpeta que chamamos **db**, tamén fóra de **src**:

- **db.json**, que simulará unha base de datos en formato *JSON*



Por que fóra de *src*?

Porque **src** **está reservado exclusivamente ao frontend**. **Vite** só debe xestionar e compilar código cliente, polo que non debería “ver” nin procesar nada relacionado co *backend*. Ademais, esta estrutura axúdanos a entender mellor a separación entre **cliente e servidor**, un concepto clave cando traballamos con *APIs* e arquitecturas modernas.

Continuamos coa instalación:

```
\CURSORVUE\proyectovue> npm install json-server@0.17.4 --save-dev

added 65 packages, and audited 223 packages in 3s

43 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

save-dev indica que *json-server* se usa só en desenvolvemento para programar e probar a aplicación, unha aproximación temporal a realidade. En produción, a aplicación conectaría cun servidor real con outro tipo de servidor de base de datos.

Por cuestións de compatibilidade con algúns prototipos convén empregar a versión indicada nos apuntes e non a máis recente (superior á 0.18). Probablemente estes problemas estarán solucionados en futuras versións.

Por outra banda imos instalar unha ferramenta ***concurrently*** que nos permite arrincar os dous servidores *dev* e *json-server* nunha soa tirada.

```
\CURSORVUE\proyectovue> npm install concurrently --save-dev

added 25 packages, and audited 108 packages in 3s

27 packages are looking for funding
  run `npm fund` for details
```

Imos a ***package.json*** e retocamos:

```
"scripts": {
  "start": "concurrently \"vite\" \"json-server—watch db/db.json—port 3000\""
}
```

De esta forma executando ***npm run start*** arrincamos os dous servidores.

```
package.json 7 ...
1 {
2   "name": "proyectovue",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "start": "concurrently \"vite\" \"json-server --watch db/db.json --port 3000\"",
9     "build": "vite build",
10    "preview": "vite preview"
11  },
12  "dependencies": {
13    "vue": "^3.5.18",
```

Para **parar** os servidores: **Ctrl+C**

```

-\\CURSOVUE\\proxectovue> npm run start

> proxectovue@0.0.0 start
> concurrently "vite" "nodemon backend/server.js --watch backend/server.js"

[1] [nodemon] 3.1.11
[1] [nodemon] to restart at any time, enter `rs`
[1] [nodemon] watching path(s): backend\\server.js
[1] [nodemon] watching extensions: js,mjs,cjs,json
[1] [nodemon] starting `node backend/server.js`
[0]
[0] VITE v7.3.0 ready in 429 ms
[0]
[0] → Local: http://localhost:5173/
[0] → Network: use --host to expose
[1] Servidor iniciado en http://localhost:3000

```

O arquivo **db.json** ten que estar xa preparado:

```

db > {} db.json > ...
1  {
2    "usuarios": [],
3    "tarefas": []
4  }

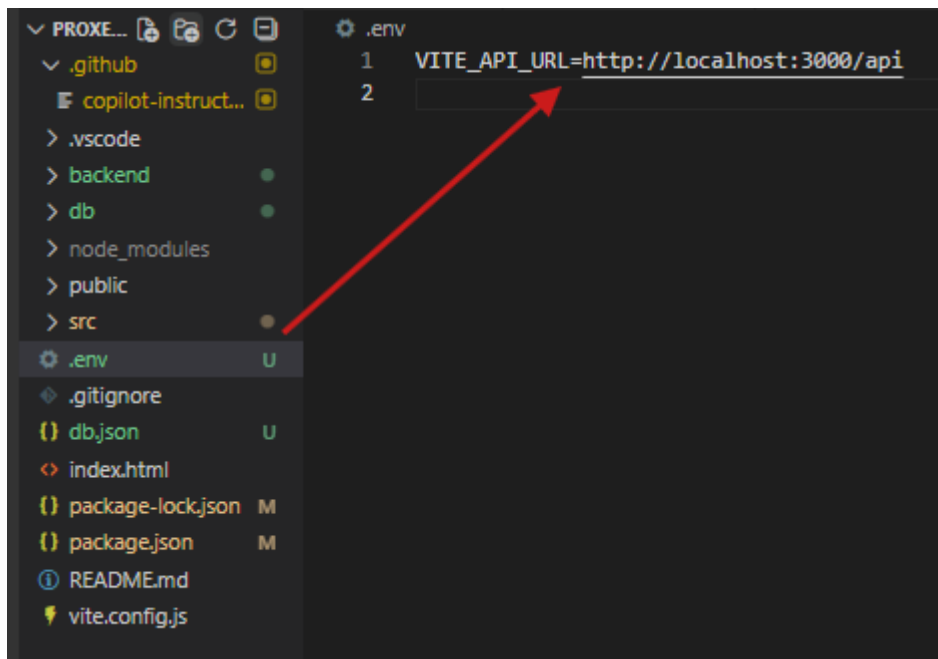
```

Con isto temos configurado xa noso **backend** simulado.

A.3. Operacións CRUD con json-server

Primeiro facemos *CRUD* cos usuarios porque son a **entidade principal da aplicación**. Unha vez dominado iso, as operacións *CRUD* das tarefas serán iguais e so temos que engadir outro recurso relacionado a través dun estado xeral para relacionar os usuarios coas súas tarefas.

A continuación, creamos un arquivo **.env** para albergar a ruta de xestión das operacións *CRUD* e centralizar a configuración da aplicación, evitando ter valores fixos no código e facilitando o cambio da *URL* da *API* entre contornas de desenvolvemento e produción.



A orde de traballo será a seguinte:

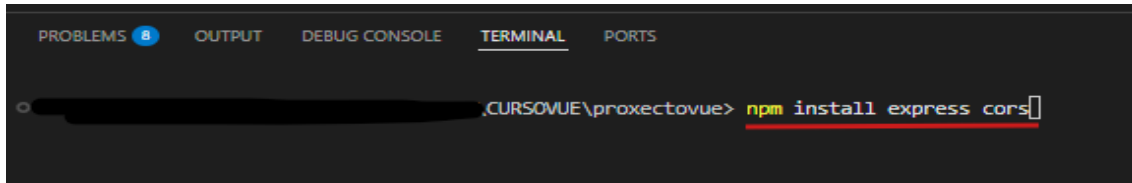
- creación do **server.js** ou arquivo que xestiona o servidor **json-server**
- codificación da *api* en *usuarios.js*
- codificación da zona de *script* de *XestionUsuarios.vue*

Paso 1. Configuración do **server.js**

Temos que instalar **express** e **cors** para crear un backend o máis próximo a realidade.

Express é un *framework* lixeiro e flexible para **Node.js** que nos permite crear servidores **web** e *APIs* de forma rápida e estruturada. Facilita a xestión de rutas, peticións *HTTP*, *middlewares* e integración con bases de datos, evitando ter que escribir todo o código de baixo nivel de *Node* puro. É a base de moitas **APIs REST** e proxectos *backend* en **JavaScript**, permitindo centralizar a lóxica do servidor e ofrecer datos ao *frontend* de forma organizada.

CORS é un mecanismo de seguridade que permite ou restrinxe que un *frontend* que se executa **nun dominio ou porto** (por exemplo, <http://localhost:5173>) poida facer peticións a un servidor **noutro dominio ou porto** (por exemplo, <http://localhost:3000>). O *middleware* **cors** para *Express* xestiona estas regras e habilita o acceso seguro desde o *frontend* ao *backend* cando se están a desenvolver aplicacións en arquitecturas separadas.



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
CURSOVUE\proyectoVue> npm install express cors

```

Chegamos a este punto aclaremos: a función de **server.js** é inicializar o servidor (**Express + json-server**) e aplica **CORS**, mentres **index.js** manterá a configuración das rutas.

A configuración do **server.js** dentro do *backend* será o que se mostra na imaxe seguinte.

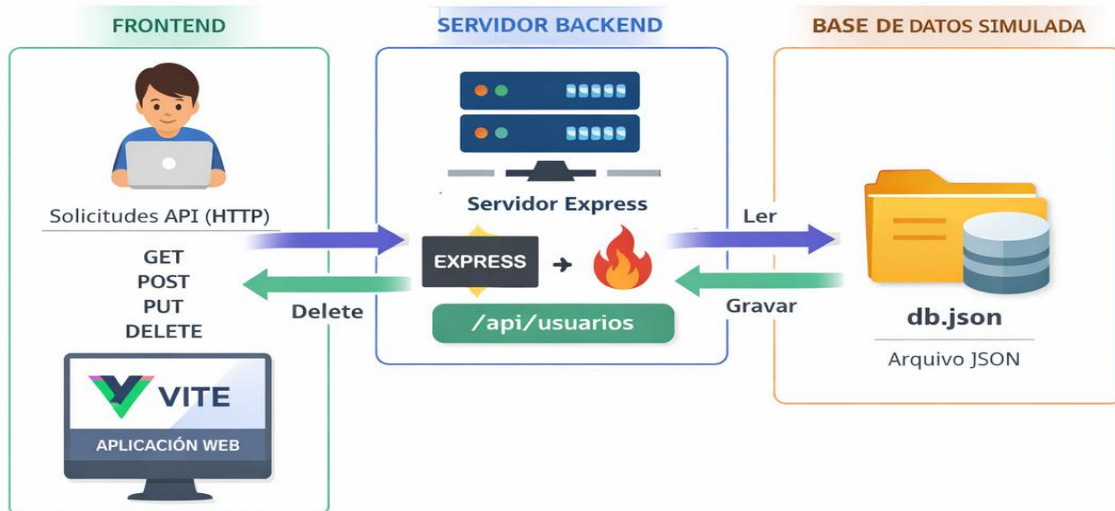
```

backend > JS server.js > ...
1 // -----
2 // Servidor para simular unha API REST con json-server que Inclúe Express para control do servidor
3 // e CORS para permitir peticións desde o frontend (Vite/localhost)
4 // -----
5
6 import express from 'express';           // Importamos Express
7 import cors from 'cors';                 // Importamos CORS
8 import jsonServer from 'json-server';    // Importamos json-server
9 import { dirname, join } from 'path';    // Importamos utilidades de path
10 import { fileURLToPath } from 'url';    // Importamos fileURLToPath para obter __dirname
11
12 const __filename = fileURLToPath(import.meta.url);
13 const __dirname = dirname(__filename);
14
15 const app = express();                   // Creamos a aplicación Express
16 const PORT = 3000;                       // Porto no que correrá o servidor
17
18 // -----
19 // Middlewares
20 // -----
21 app.use(cors());                         // Permite peticións de calquera orixe (CORS) evitando problemas co frontend
22
23 // Creamos un router de json-server apuntando ao ficheiro db.json
24 const router = jsonServer.router(join(__dirname, '..', 'db', 'db.json'))
25
26 // Podemos engadir middlewares de json-server se queremos (logs, retrasos, etc.)
27 app.use(jsonServer.defaults())           // Middlewares por defecto (logger, static files...)
28
29 app.use('/api', router)                  // Todas as rutas da API estarán baixo /api // /api é só o "nome da ruta pública".
30 // Non ten que coincidir coa estrutura de carpetas do backend,
31 // é máis unha convención para organizar as URLs do teu API.
32
33
34 // GET http://localhost:3000/api/usuarios
35 // POST http://localhost:3000/api/usuarios
36 // PUT http://localhost:3000/api/usuarios/1
37 // DELETE http://localhost:3000/api/usuarios/1
38
39 // -----
40 // Arranque do servidor
41 // -----
42 app.listen(PORT, () => {
43   console.log(`Servidor iniciado en http://localhost:${PORT}`)
44 })

```

Importante: este **archivo está no almacén da UD4**. Pero as modificacións posteriores que fagamos xa corren da túa conta.

O esquema de funcionamento resúmese na imaxe seguinte.



Temos agora que configurar o arranque de **server.js**. Podemos detelo (**Ctrl + C**) e arrincalo manualmente cada vez que fagamos un cambio, pero iso resulta incómodo.

No período de desenvolvemento é habitual usar **nodemon**, unha ferramenta que observa os cambios no código e **reinicia automaticamente o servidor** cando detecta modificacións. Isto axuda a iterar rápido sen ter que parar e iniciar manualmente cada vez que editamos **usuarios.js** ou **tarefas.js**.

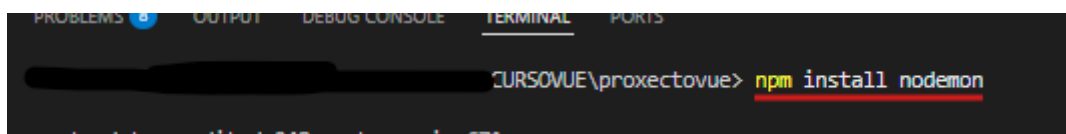
Podemos ir un paso máis alá, e integralo en *concurrently* no arquivo *package.json*, para que inclúa **nodemon** no backend, de modo que cada vez que cambiemos o código do servidor, este **se recargue automaticamente** sen ter que reinicialo manualmente, facendo o desenvolvemento máis áxil executando só unha vez **npm run start**

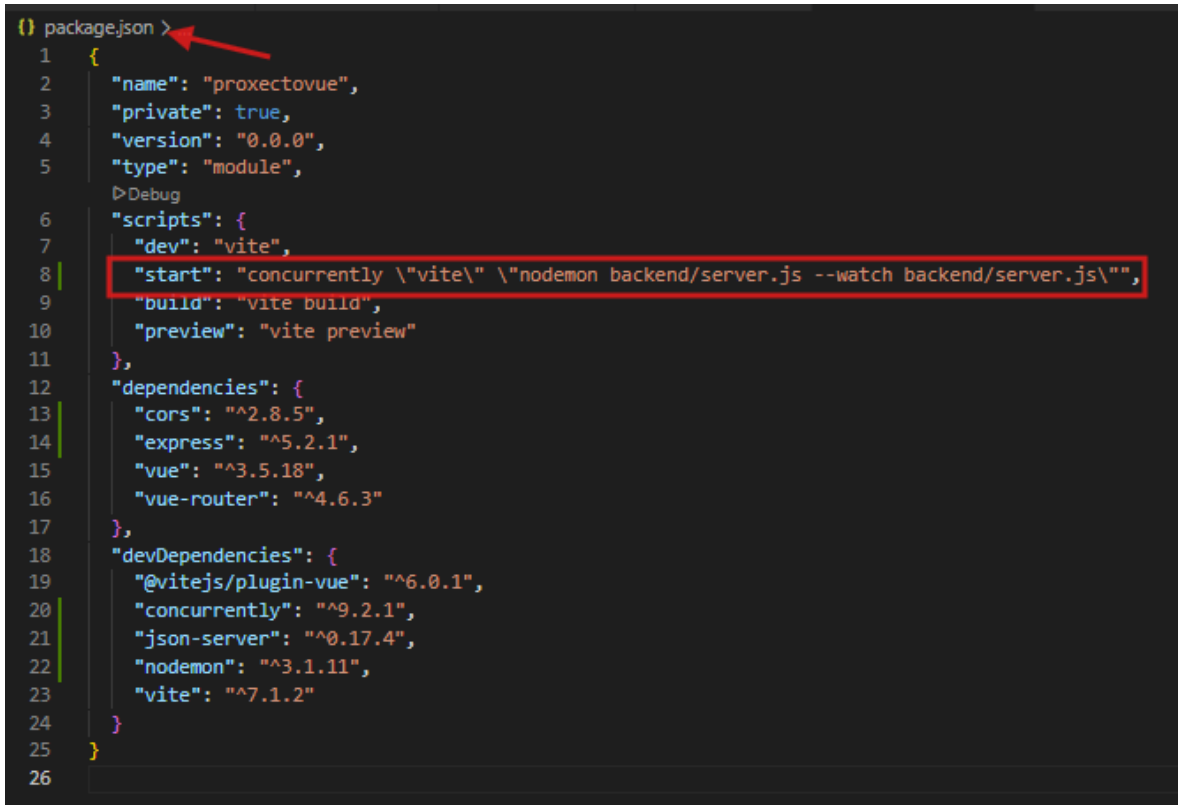
```

"scripts": {
  ...
  "start": "concurrently \"vite\" \"nodemon backend/server.js--watch backend/server.js\"",
  ....
},

```

Non esquecemos de instalar **nodemon**.



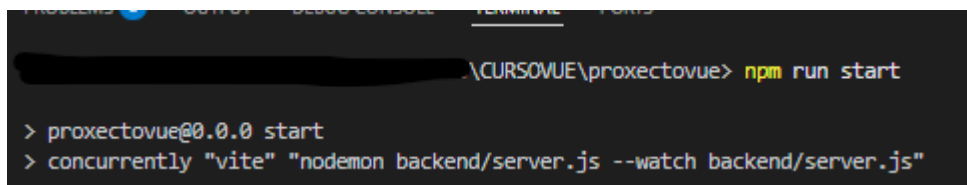


```

() package.json >
1  {
2    "name": "proxectovue",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "start": "concurrently \"vite\" \"nodemon backend/server.js --watch backend/server.js\"",
9      "build": "vite build",
10     "preview": "vite preview"
11   },
12   "dependencies": {
13     "cors": "^2.8.5",
14     "express": "^5.2.1",
15     "vue": "^3.5.18",
16     "vue-router": "^4.6.3"
17   },
18   "devDependencies": {
19     "@vitejs/plugin-vue": "^6.0.1",
20     "concurrently": "^9.2.1",
21     "json-server": "^0.17.4",
22     "nodemon": "^3.1.11",
23     "vite": "^7.1.2"
24   }
25 }
26

```

Arrincamos e xa temos o servidor funcionando.



```

.\CURSORVUE\proxectovue> npm run start

> proxectovue@0.0.0 start
> concurrently "vite" "nodemon backend/server.js --watch backend/server.js"

```

Unha cuestión que hai que comentar é esta liña do arquivo **server.js**

app.use('/api', router)

Se non existe a carpeta **api**, por qué chamamos a **/api/usuarios**?

Porque **/api** non é unha carpeta, **é unha ruta virtual** definida en **Express** e as rutas **HTTP** non dependen da estrutura de carpetas, senón do **router**. Esta é a clave que convén ter clara.

Esta é a clave que convén ter clara, así podemos ter rutas como:

/api/usuarios

/api/tareras

ambas dúas son rutas lóxicas, non carpetas físicas, aínda que poderíamos habelas creado, non é necesario.

O resto do código de **server.js** é autoexplicativo, sendo importacións, instancias e declaración de constantes.

Imos ao *frontend*.

Paso 2. Quédamos conectar a zona de **script** de *XestionUsuarios.vue* co *backend*.

Imos a ir, un por un, para que vaias modificando o que temos na zona de *script* e ir probando.

Nas unidades anteriores traballabamos con datos gardados na memoria do navegador, polo que non era necesario esperar resposta dun servidor.

Tamén observarás que engadimos as funcións asíncrona, **async** xa que a función pode tardar en rematar, porque espera unha resposta do servidor, e ao facela **asíncrona** evitamos que bloquee a aplicación mentres espera, **await**, o resultado **sen conxelar a interface**. Nas unidades anteriores ao traballar en memoria do navegador non tiñamos ese problema.

Tampouco debemos esquecernos das seguintes importacións que son:

- **Axios** a librería que se comunica con backend
- **API_URL** recorda que usamos **.env** para manexar as **URL**
- **ref** e **reactive** de acordo a complexidade a usar.

```
<script setup>
import { ref, reactive } from "vue";
import axios from "axios";
import { onMounted } from "vue";

//URL dende .env
const API_URL = import.meta.env.VITE_API_URL;

const usuarios = ref([]); // Array que se conectará co json-server

const novoUsuario = reactive({
  dni: "",
  nome: "",
  correo: "",
  provincia: "",
  activo: false,
  tipoCuenta: "",
});

const usuarioSeleccionado = ref(null);
const tarefasUsuario = ref([]);
```

- **GET – obter usuarios**

Comezamos coa **primeira operación CRUD**, que é obter o listado de usuarios da base de datos **GET**. Ao principio está o nome está en cor gris, iso significa que aínda non se está usando. Pero nunha aplicación deste estilo nos interesa que nos mostre os usuarios da base de datos cada vez que carguemos o compoñente. Recorda que isto facíámolo con **onMounted()**. Pois entón, só temos en engadir o seguinte:

```

1  const usuarioSeleccionado = ref(null);
2  const tarefasUsuario = ref([]);
3
4  onMounted(() => {
5    cargarUsuarios();
6  });
7
8  // Funcion GARGAR USUARIOS
9  async function cargarUsuarios() {
10    try {
11      const res = await axios.get(`${API_URL}/usuarios`);
12      usuarios.value = res.data;
13    } catch (error) {
14      console.error("Erro ao cargar usuarios", error);
15    }
16  }

```

De momento ao non haber usuarios en **db.json** non mostrará nada, pero imos a **segunda operación CRUD, POST**, que é gardar usuarios.

- **POST – engadir usuarios**

Esta función fai dúas cousas, gardar un usuario se non está ou modifícalo nos seus datos se o editamos, aproveitando o mesmo botón para ambas cousas.

```

// CREAR OU MODIFICAR USUARIO
async function gardarUsuario() {
  try {
    if (usuarioSeleccionado.value) {
      await axios.put(
        `${API_URL}/usuarios/${usuarioSeleccionado.value.id}`,
        novoUsuario
      );
    } else {
      await axios.post(`${API_URL}/usuarios`, novoUsuario);
    }

    await cargarUsuarios();
    limpiarFormulario();
  } catch (error) {
    console.error("Erro ao gardar usuario", error);
  }
}

```

¡molo probar.

Abrimos a aplicación e introducimos uns datos:

```

{
  dni: "55555555K",
  nome: "Margarita Flor Branca",
  correo: "branca@mail.com",
  provincia: "Lugo",
  activo: true,
  tipoCuenta: "particular",
  id: 1
}

```

Se observamos o código, cando gardamos un usuario (*POST*) carga a táboa chamando a *cargarUsuarios* e limpa o formulario (opcional) chamando a función *limparFormulario()* que como son datos da memoria do navegador non é necesario *await*.

```

// Limpar formulario
function limpiarFormulario() {
  novoUsuario.dni = "";
  novoUsuario.nome = "";
  novoUsuario.correo = "";
  novoUsuario.provincia = "";
  novoUsuario.activo = false;
  novoUsuario.tipoCuenta = "";
  usuarioSeleccionado.value = null;
  tarefasUsuario.value = [];
}

```

- **PUT – modificar usuarios**

Por outra banda, o mesmo botón **Gardar**, serve **para modificar os datos do usuario (PUT)** tendo en conta que **json-server** garda un usuario con un **id**.

O botón coa **icona do lapis**, carga os datos no formulario segundo o **index**. Neste caso imos pasar o usuario de non – activo a activo, e imos poñelo de tipo empresa.

```
function editarUsuario(index) {
  const usuario = usuarios.value[index];
  Object.assign(novoUsuario, usuario); // Copiamos os datos do usuario ao formulario
  usuarioSeleccionado.value = usuario; // Marcamos o usuario como seleccionado para editar
  tarefasUsuario.value = []; // Sen tarefas de exemplo
}
```

```
{
  "dni": "00000000T",
  "nome": "Xoan Sen Nome",
  "correo": "xoan@mail.com",
  "provincia": "Pontevedra",
  "activo": false,
  "tipoCuenta": "empresa",
  "id": 1
}
```

- **DELETE – eliminar usuarios**

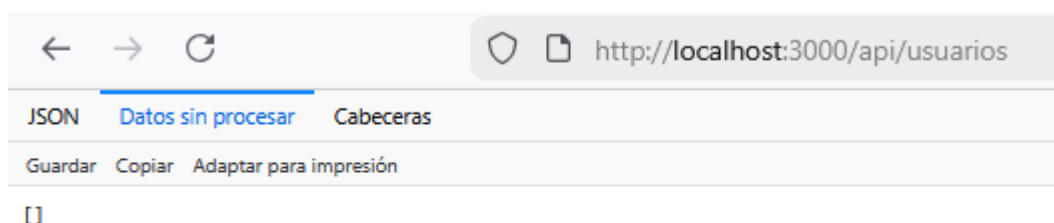
A última operación **CRUD** será eliminar usando como botón a icona do símbolo do cubo de lixo, pero editando antes. Como se ve o código indica que a partires do **id** que aporta **json-server** eliminamos o usuario.

Nas seguintes imaxes podemos velo código así como o resultado da operación delete con **axios**.

```

8
9 function eliminarUsuario(index) {
10   const usuario = usuarios.value[index];
11   axios.delete(
12     `${API_URL}/usuarios/${usuario.id}`
13   ).then(() => {
14     cargarUsuarios();
15   })
16   .catch((error) => {
17     console.error("Erro ao eliminar usuario", error);
18   });
19 }

```



Hai un punto que apenas nos fixamos que é o **template**, o cal ten un par de cousas interesantes especialmente na zona da táboa.

Imos comentala pero antes mostramos o código.

```

</thead>
<tbody>
  <tr v-for="(u, index) in usuarios" :key="index">
    <td>{{ index + 1 }}</td>
    <td style="text-align: center">{{ u.dni }}</td>
    <td>{{ u.nome }}</td>
    <td>{{ u.correo }}</td>
    <td>{{ u.provincia }}</td>
    <td style="text-align: center">{{ u.activo ? "✅" : "❌" }}</td>
    <td>{{ u.tipoCuenta }}</td>
    <td style="text-align: center">
      <button @click="editarUsuario(index)" title="Editar">✎</button>
      <button @click="eliminarUsuario(index)" title="Eliminar">🗑️</button>
    </td>
  </tr>
</tbody>

```

Se nos fixamos en **u.activo** actúa como una directiva **v-if** no cal, se o campo activo do usuario é **True** entón mostra a etiqueta verde, pero si é **False** mostra a aspa vermella.

Ademais, no botón do formulario tamén hai un punto interesante que volve a ser a directiva **v-bind** ou **":"** que son o mesmo.

```

65   </div>
66   <button
67     type="submit"
68     class="btn-guardar"
69     :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''"
70   >
71     Gardar
72   </button>

```

Neste caso a acción do botón deshabilitase se o campo **dni** ou o campo **nome** ou ambos, están baleiros.

Isto permítenos controlar o **comportamento da interface en función do estado dos datos** introducidos polo usuario, evitando envíos incompletos do formulario.

A.4. Estado global e comunicación entre vistas con Pinia

Ata o de agora estivemos traballando cos datos de forma local dentro de cada compoñente. Isto funciona ben mentres toda a lóxica está nun único compoñente, pero comeza a quedar limitado cando necesitamos **compartir información entre vistas distintas e o proxecto escala en complexidade**.

No noso caso, ao seleccionar un usuario en *XestiónUsuarios.vue* precisamos **gardar o seu id e o seu nome** para poder xestionar as tarefas asociadas a ese usuario en *XestiónTareas.vue*, que se carga noutra vista diferente.

Como estes compoñentes **non teñen relación pai-fillo directa**, que é o habitual en proxectos complexos, non é práctico nin recomendable usar **props** nin **emits**. En compoñentes onde esa **xerarquía non está clara** ou simplemente **non existe**, **convén usar estados centralizados**.

Un **estado centralizado** actuará como **unha zona común** onde gardar os datos do usuario seleccionado, de forma que poidan ser utilizados por distintos compoñentes da aplicación. Deste xeito:

- *XestiónUsuarios.vue* poderá actualizar o usuario seleccionado.
- *XestiónTareas.vue* poderá **ler ese mesmo usuario** en calquera momento.
- Evitamos duplicar datos e mantemos a aplicación máis organizada e escalable.

Este enfoque é o que se emprega habitualmente **en aplicacións reais cando diferentes partes da aplicación necesitan acceder á mesma información**, independentemente da vista ou compoñente no que se atopen cando nos movemos entre os compoñentes.

Para estas situacións imos empregar **Pinia**, o sistema oficial de xestión de estado de *Vue*.

Pinia permítenos definir un **estado global**, accesible desde calquera compoñente da aplicación, independentemente da vista na que se atope.

Esta ferramenta substitúe a outras solucións máis sinxelas como **provide** e **inject**, ambas ferramentas permiten compartir datos entre compoñentes con xerarquía sen usar **props**, pero **non está pensado para compartir estado entre vistas, pero sobre todo para estados complexos**.

Por este motivo, en aplicacións reais empréganse solucións de estado global como **Pinia** que é a ferramenta máis usada neste ámbito

Empecemos con **Pinia**.

Paso 1. Instalación

```
\CURSORVUE\proyectovue> npm install pinia
added 13 packages, and audited 247 packages in 3s
49 packages are looking for funding
```

Paso 2. Rexistro de *Pinia* na aplicación

Unha vez instalada, debemos rexistrar *Pinia* na aplicación principal. Para iso modificamos o arquivo **main.js**

```
src > JS main.js > ...
1 import { createApp } from 'vue'
2 | import { createPinia } from 'pinia'
3 import './style.css'
4 import App from './App.vue'
5 import router from './router'
6
7 const pinia = createPinia()
8
9 createApp(App).use(router).use(pinia).mount('#app')
```

Paso 3. Estrutura básica do estado global

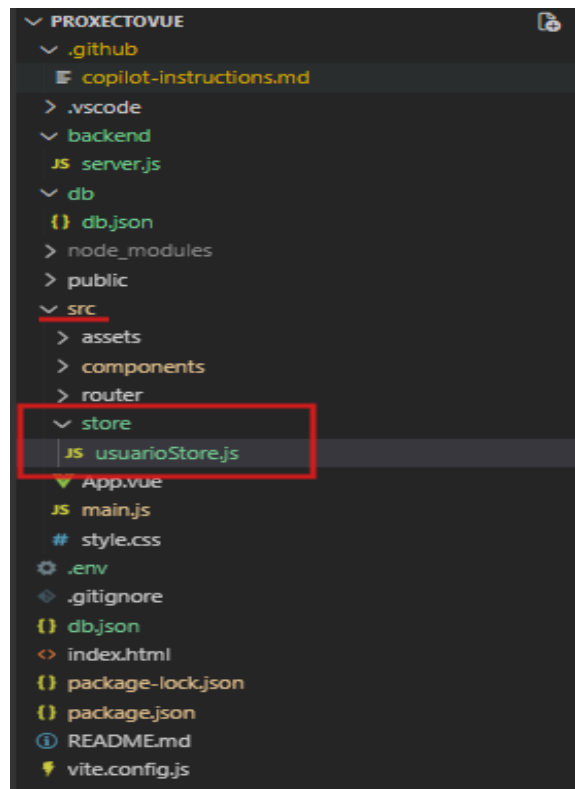
Un **store** é simplemente un **módulo onde centralizamos datos e funcións** relacionadas con eses datos.

Creamos o **store** onde almacenaremos o **id** e o **nome**. Hai que ter en conta que interésanos almacenar os datos só ao **seleccionUsuario** e tamén para **limparUsuario** que é o noso contexto de traballo.

Pódese pensar que crear o **store** en **src** pode supor un problema de seguridade dependendo da sensibilidade dos datos: *nome*, *id*.

Outra cousa sería gardar *tokens*, *claves API* ou *contrasinais* que non é o caso e iso se validaría e/ou **gardaríase no backend**.

En resumo, *Pinia* é un almacén de estado do *frontend*, non un sistema de seguridade que nunca debe almacenar datos sensibles.



No noso caso, imos gardar o **usuario seleccionado** nun estado global con **Pinia**. Así, cando neste caso se selecciona na listaxe un usuario, gardaremos o seu *id* e nome no **store**. De esta forma, calquera **outro compoñente ou vista poderá acceder a estes datos** sen necesidade de *props* ou eventos, mantendo a aplicación organizada e facilitando a xestión de tarefas asociadas a cada usuario, e dicir

1. Seleccionas un usuario en *XestionUsuarios*
 2. Garda os datos no **usuarioStore.js**
 3. En *XestionTareas* colle os datos do *store*
1. Empecemos coa configuración de **usuariosStore.js**. O código xa está no almacén comentado pero imos recordalo. A primeira parte consiste na importación do módulo *Pinia* e na definición dunha variable **Store** que conteña os dous datos que nos interesan, o **id** e o **nome** do usuario do cal imos xestionar a súas tarefas. Despois temos dúas funcións: a acción que toman os valores en cuestión, **seleccionarUsuario** e a que borran eses valores **limparUsuario**, cando abandonamos o compoñente **XestionTareas.vue**. En resumo, definimos o estado (**id e nome**) e as accións asociadas.

```
src > store > JS usuarioStore.js > ...
1  import { defineStore } from "pinia";
2
3  // Definimos a store para xestionar o usuario seleccionado
4  export const useUsuarioStore = defineStore("usuario", {
5    // Estado inicial da store
6    state: () => ({
7      id: null,
8      nome: null,
9    }),
10
11   // Acciones para modificar o estado da store
12   actions: {
13     seleccionarUsuario(usuario) {
14       this.id = usuario.id;
15       this.nome = usuario.nome;
16     },
17
18     limparUsuario() {
19       this.id = null;
20       this.nome = null;
21     },
22   },
23 });
24
```

2. Engadimos os seguintes códigos en *XestiónUsuarios.vue*, o que nos permiten **facér a chamada ao store para gardar o usuarios seleccionado**. Basicamente estará na función **gardarUsuario** e na función **editarUsuario** que son as dúas que poden requirir que se garden os datos do usuario para usalos en outros compoñentes, neste caso en *XestiónTarefas.vue*. Non esquecer que hai importar tamén o **Store**. Para facilitar a lectura tomamos **só as zonas de código que se engaden**. As modificación nas zonas de importación e declaracións:

```
src > components > ▼ XestiónUsuarios.vue > {} script setup
108 </template>
109
110 <script setup>
111 import { ref, reactive, onMounted } from "vue";
112 import axios from "axios";
113 import { useUsuarioStore } from '../store/usuarioStore.js';
114
115 // Instanciamos a store de usuario
116 const usuarioStore = useUsuarioStore();
117
118 //URL dende .env
119 const API_URL = import.meta.env.VITE_API_URL;
120
121 const usuarios = ref([]); // Array que se conectará co json-server
122
123 const novoUsuario = reactive({
```

O que engadimos en *gardarUsuario()*

```
// CREAR OU MODIFICAR USUARIO

async function gardarUsuario() {
  try {
    if (usuarioSeleccionado.value) {
      await axios.put(
        `${API_URL}/usuarios/${usuarioSeleccionado.value.id}`,
        { ...novoUsuario }
      );
    } else {
      await axios.post(`${API_URL}/usuarios`, novoUsuario);
    }

    // Actualizamos no Store o estado do usuario seleccionado store ou o novo usuario
    usuarioStore.seleccionarUsuario(usuarioSeleccionado.value || novoUsuario);

    await cargarUsuarios();
    limpiarFormulario();
  } catch (error) {
    console.error("Erro ao gardar usuario", error);
  }
}
```

O código engadido en *editarUsuario*.

```
function editarUsuario(index) {
  const usuario = usuarios.value[index];
  Object.assign(novoUsuario, usuario); // Copiamos os datos do usuario ao formulario
  usuarioSeleccionado.value = usuario; // Marcamos o usuario como seleccionado para editar
  usuarioStore.seleccionarUsuario(usuario); // Actualizamos o Store co usuario seleccionado

  tarefasUsuario.value = []; // Sen tarefas de exemplo
}
```

E cando eliminamos o usuario, eliminamos o **Store**

```
function eliminarUsuario(index) {
  const usuario = usuarios.value[index];
  axios.delete(
    `${API_URL}/usuarios/${usuario.id}`
  ).then(() => {
    cargarUsuarios();
    // Se o usuario eliminado era o seleccionado, limpamos o formulario e o Store
    if (usuarioSeleccionado.value && usuarioSeleccionado.value.id === usuario.id) {
      limpiarFormulario();
      usuarioStore.limparUsuario();
    }
  })
  .catch((error) => {
    console.error("Erro ao eliminar usuario", error);
  });
}
```

3. Quédanos as modificacións en **XestiónTareas.vue**. Este é o compoñente que fai uso dos datos gardados no *Store*.

Empeza coa importación e declaración da variable.

```
<script setup>
import { ref, onMounted } from 'vue'
import { useUsuarioStore } from '../store/usuarioStore.js'

// Estes datos virán do store ou das peticións HTTP na UD4
const usuario = ref(null) // Ex: { id: 3, nome: 'María' }
const tarefas = ref([])

// Instanciamos a store de usuario
const usuarioStore = useUsuarioStore()

// Cargar Tareas do usuario seleccionado
onMounted(() => {
  usuario.value = {
    id: usuarioStore.id,
    nome: usuarioStore.nome,
  }
})
```

Chegados a este punto xa podemos ver o funcionamento do **Store**.

Pero antes, facemos unha pequeno cambio na zona do **template** en **XestionTarefas.vue** de maneira que cargue, de momento o *id* e o *nome* do usuario que seleccionamos

```
src > components > XestionTarefas.vue > ...
1 <template>
2 <div class="xestion-tarefas">
3   <h3 v-if="usuarioStore.id">
4     Código Usuario: {{ usuarioStore.id }} === Tarefas de {{ usuarioStore.nome }}
5   </h3>
6
7   <p v-else>Non hai usuario seleccionado</p>
8
9   <!-- FORMULARIO NOVA TAREFA -->
10  <form @submit.prevent="engadirTarefa" class="form-tarefa">
11    <input v-model="novaTarefa.dataLimite" type="date" required />
```

Así, cando cargamos un usuario en **XestionUsuarios.vue**

The screenshot shows the 'Xestión de usuarios' form with the following fields: DNI/CIF (55555555K), Nome (Margarita Flor Branca), Correo (branca@mail.com), and Provincia (Lugo). The 'Activo' checkbox is checked. Below the form is a table titled 'Listaxe de usuarios' with the following data:

#	DNI/CIF	Nome	Correo	Provincia	Activo	Tipo de conta	Accións
1	55555555K	Margarita Flor Branca	branca@mail.com	Lugo	☒	particular	

E vamos agora a **XestionTarefas.vue**

The screenshot shows the 'Xestión de tarefas' form with the following fields: 'Código Usuario: 1 === Tarefas de Margarita Flor Branca' (highlighted with a red box), 'dd / mm / aaaa' (date picker), and 'Título ou descripción da tarefa' (text area). Below the form is a button 'Grabar tarefa' and a message 'Este usuario non ten tarefas asignadas.'

Temos os datos do **id** e o **nome** cargados. Xa só nos queda gardar e xestionar as tarefas de cada usuario.

Só traballaremos as seguintes funcións:

- Listar as tarefas de cada usuario: **cargarTarefas**
- Gardar novas tarefas para cada usuario: **grabarTarefas**
- Modificar o estado das tarefas: **completarTarefa**

Empecemos coas **declaracións**, despois con **cargarTarefas** en **onMounted()** para que cando se monte o compoñente cargue as tarefas do usuario, e **onUnmounted()** para limpar o Store cando salimos do compoñente.

```

55
56 <script setup>
57 import { ref, onMounted, onUnmounted } from "vue";
58 import axios from "axios";
59 import { useUsuarioStore } from "../store/usuarioStore.js";
60
61 const API_URL = import.meta.env.VITE_API_URL;
62 // Estes datos virán do store ou das peticións HTTP na UD4
63
64 const usuario = ref(null); // Ex: { id: 3, nome: 'María' }
65 const tarefas = ref([]);
66
67 // Instanciamos a store de usuario
68 const usuarioStore = useUsuarioStore();
69
70 // Nova tarefa
71 const novaTarefa = ref({
72   titulo: "",
73   dataLimite: "",
74 });
75
76
77 // Cargar Tarefas do usuario seleccionado
78 onMounted(async () => {
79   // Aquí faríamos unha petición HTTP para obter as tarefas do usuario
80   if (!usuarioStore.id) return;
81
82   // esto podría facerse ou directamente desde a store ou ben con computed
83   usuario.value = {
84     id: usuarioStore.id,
85     nome: usuarioStore.nome,
86   };
87
88   // Petición HTTP para obter as tarefas do usuario seleccionado
89   try{
90     const res = await axios.get(
91       `${API_URL}/tarefas?usuarioId=${usuario.value.id}`
92     );
93     tarefas.value = res.data;
94   } catch (error) {
95     console.error("Erro ao cargar as tarefas:", error)
96   }
97 }
98 );
99
100 // Limpar datos ao desmontar o compoñente
101 onUnmounted(() => {
102   usuarioStore.limparUsuario();
103   tarefas.value = [];
104 });
105
106

```


Podemos ver as importacións, as declaración dende o **Store** e cando cargamos as tarefas de usuario cando se monta o compoñente e limpar o **Store** cando saímos do compoñente, algo que moitos programadores non fan e poden crear problemas cos datos.

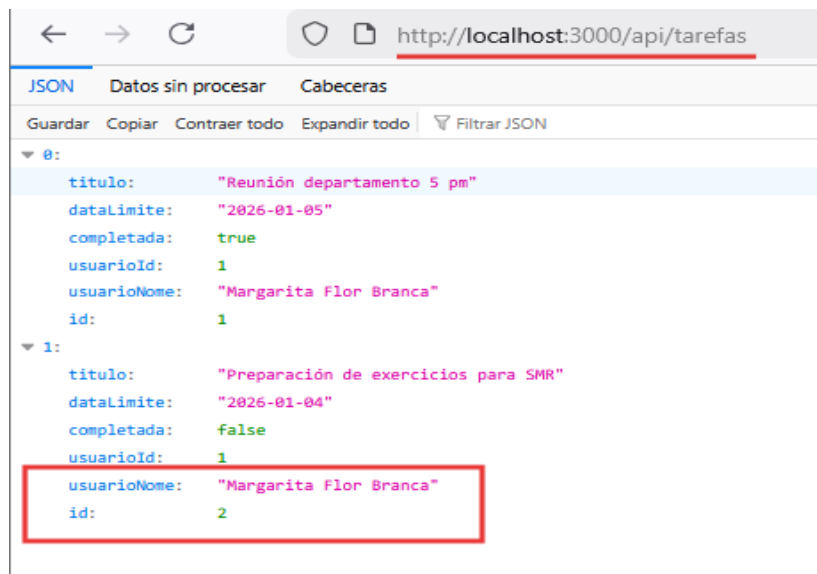
A segunda modificación é codificar o módulo **cargarTarefas** que carga as tarefas do usuario que temos no **Store**. Usamos **POST**, pero o importante é ver como asociamos a tarefa creada ao usuario pasado polo **Store**.

```
// Crear tarefa (POST máis adiante con axios)
async function engadirTarefa() {
  try {
    // Construír obxecto nova tarefa
    const nova = {
      titulo: novaTarefa.value.titulo,
      dataLimite: novaTarefa.value.dataLimite,
      completada: false,
      usuarioId: usuarioStore.id,
      usuarioNome: usuarioStore.nome,
    };
    // Petición POST para gardar a nova tarefa
    const res = await axios.post(`${API_URL}/tarefas`, nova);
    // Engadir a tarefa á lista local e cargue a taboa
    tarefas.value.push(res.data);

    novaTarefa.value.titulo = "";
    novaTarefa.value.dataLimite = "";
  } catch (error) {
    console.error("Erro ao gardar tarefa", error);
  }
}

// Marcar tarefa como completada (PUT máis adiante)
```

Se vemos no **db.json**



E na táboa:

Código Usuario: 1 === Tarefas de Margarita Flor Branca

08 / 01 / 2026

Reunión avaluación 2º DAM as 4

Grabar tarefa

ID	Tarefa	Data límite	Estado	Acciões
1	Reunión departamento 5 pm	2026-01-05	✓	✓
2	Preparación de ejercicios para SMR	2026-01-04	✗	✓

Tras engadir tarefa:

Código Usuario: 1 === Tarefas de Margarita Flor Branca

dd / mm / aaaa

Título ou descripción da tarefa

Grabar tarefa

ID	Tarefa	Data límite	Estado	Acciões
1	Reunión departamento 5 pm	2026-01-05	✓	✓
2	Preparación de ejercicios para SMR	2026-01-04	✗	✓
3	Reunión avaluación 2º DAM as 4	2026-01-08	✗	✓

Finalmente, quedáanos o código que tras pulsar o botón en **Acciões** pasa o estado da tarefa de pendente a feito. Neste caso, tamén hai que botarlle un vistazo ao **template** para entender como a zona de **script** conecta coa zona **template**.

```

</thead>
<tbody>
  <tr v-for="(t, index) in tarefas" :key="index">
    <td style="text-align: center">{{ index + 1 }}</td>
    <td>{{ t.titulo }}</td>
    <td style="text-align: center">{{ t.dataLimite }}</td>
    <td style="text-align: center">
      {{ t.completada ? "✅" : "❌" }}
    </td>
    <td style="text-align: center">
      <button @click="completarTarefa(index)" :disabled="t.completada">
        ✓
      </button>
    </td>
  </tr>
</tbody>

```

Si a *t.completada* é **True** pon a etiqueta verde, pero si está a **False** enton pon a aspa vermella.

Ademais, algo **importante**, unha vez máis temos unha directiva **v-bind** ou ":" no botón, onde si *t.completada* está a **True** deshabilita o botón xa que non ten sentido. Podería ser que quixésemos cambiar o estado pero iso xa dependería dos requirimentos do usuario final.

Quédanos ver o código que manexa o **template**:

```

// Marcar tarefa como completada (PUT máis adiante)
async function completarTarefa(index) {
  const tarefa = tarefas.value[index];

  try {
    const res = await axios.patch(
      `${API_URL}/tarefas/${tarefa.id}`,
      { completada: true }
    );

    tarefas.value[index] = res.data;
  } catch (error) {
    console.error("Erro ao actualizar tarefa", error);
  }
}

```

Como vemos, si a tarefa existe búscase e pon o campo **completada** a **True**. Cando se carga a táboa se o campo da tarefa está **True** ponse ca etiqueta verde como dixemos, ademais de deshabilitar o botón, e se encontra a **False** pon a aspa vermella e habilita o botón.

Ata aquí teríamos unha app sinxela pero completa xa funcional. Só nos quedaría configurar un servidor, compilar a aplicación con **npm run build** e subir todo ao servidor. Pero sería outra historia.

A.5. Xestión de estado baseada en parámetros de ruta

Ata agora estamos a usar **Pinia** para gardar o usuario seleccionado, o cal está perfecto para entender o concepto de estado global. Pero nesta sección imos cambiar o punto de vista: xa non é o usuario quen “vive” na aplicación, senón que é a **URL a que define o contexto da vista**.

De feito, nas aplicacións reais tamén é moi habitual que:

- A vista dependa da **URL**
- Se poida **recargar a páxina** sen perder o contexto
- Se poida **compartir unha URL** directamente (por exemplo, tarefas do usuario 3)

Para isto teríamos que:

1. No arquivo **index.js** establecer a seguinte ruta con **parámetro**, onde **id** sería o **id** do usuario.

```
const routes = [
  { path: '/', name: 'inicio', component: Inicio },
  { path: '/usuarios', name: 'usuarios', component: XestionUsuarios },
  { path: '/tarefas', name: 'tarefas', component: XestionTarefas },
  { path: '/sobrenos', name: 'sobrenos', component: SobreNos },
  { path: '/avisoLegal', name: 'avisoLegal', component: AvisoLegal },
  { path: "/tarefas/:id", name: "xestionTarefas", component: XestionTarefas }
```

2. No compoñente **XestionUsuarios.vue** faríamos a seguinte modificación, soamente na zona do **template** concretamente na zona da táboa onde engadimos a ruta

```
<td style="text-align: center;">
  <button @click="editarUsuario(index)" title="Editar"></button>
  <button @click="eliminarUsuario(index)" title="Eliminar"></button>
  <router-link
    :to="{ name: 'xestionTarefas', params: { id: u.id } }"
    class="btn"
  >
    Tarefas
  </router-link>
</td>
</tr>
```

A vista quedaría así:

Listaxe de usuarios							
ID	DNI/CIF	Nome	Correo	Provincia	Activo	Tipo de conta	Accións
1	55555555K	Margarita Flor Branca	branca@mail.com	Lugo	☒	particular	   Tarefas

3. No compoñente *XestiónTarefas.vue* e onde van o maior número de modificacións. Tampouco moitas.

- a. En primeiro lugar na zona de declaracións. cando accedemos á vista ***XestiónTarefas.vue*** a través dun enlace de navegación, entramos nun escenario distinto: **estamos cambiando de ruta.**

```

58 <script setup>
59 import { ref, onMounted, onUnmounted } from "vue";
60 import axios from "axios";
61 import { useUsuarioStore } from "../store/usuarioStore.js";
62 import { useRoute } from "vue-router";
63 import { watch } from "vue";
64
65 const route = useRoute();
66
67 const API_URL = import.meta.env.VITE_API_URL;
68 // Estes datos virán do store ou das peticións HTTP na UD4

```

- b. Por outro lado, **comentamos os códigos** de ***onMounted*** e de ***onUnmounted***, que son substituídos por ***watch***

```

82 // Watch para cargar datos do usuario ao cambiar a ruta
83 watch(() => route.params.id,
84   async (idUserio) => {
85     if (!idUserio) return;
86
87     try {
88       // Cargar o usuario
89       const resUsuario = await axios.get(
90         `${API_URL}/usuarios/${idUserio}`
91       );
92
93       usuario.value = {
94         id: resUsuario.data.id,
95         nome: resUsuario.data.nome,
96       };
97
98       // Cargar tarefas do usuario
99       const resTarefas = await axios.get(
100         `${API_URL}/tarefas?usuarioId=${idUserio}`
101       );
102       tarefas.value = resTarefas.data;
103
104     } catch (error) {
105       console.error("Erro ao cargar datos", error);
106     }
107   },
108   // Opcional: cargar inmediatamente ao montar o compoñente
109   { immediate: true }
110 );
111 /*
112 // Cargar Tarefas do usuario seleccionado
113 onMounted(async () => {
114   // Aquí faríamos unha petición HTTP para obter as tarefas do usuario
115   if (usuarioStore.id) return;

```

watch empregamos esta función para **detectar cambios no parámetro *id*** da ruta. Isto permítenos **reaccionar automaticamente** cada vez que se selecciona un usuario distinto desde ***XestiónUsuarios.vue*** cando pulsamos o **botón tarefas**.

O que está “mirando” **watch** son os cambios en ***params.id***, e dicir, dos cambios da **URL**.

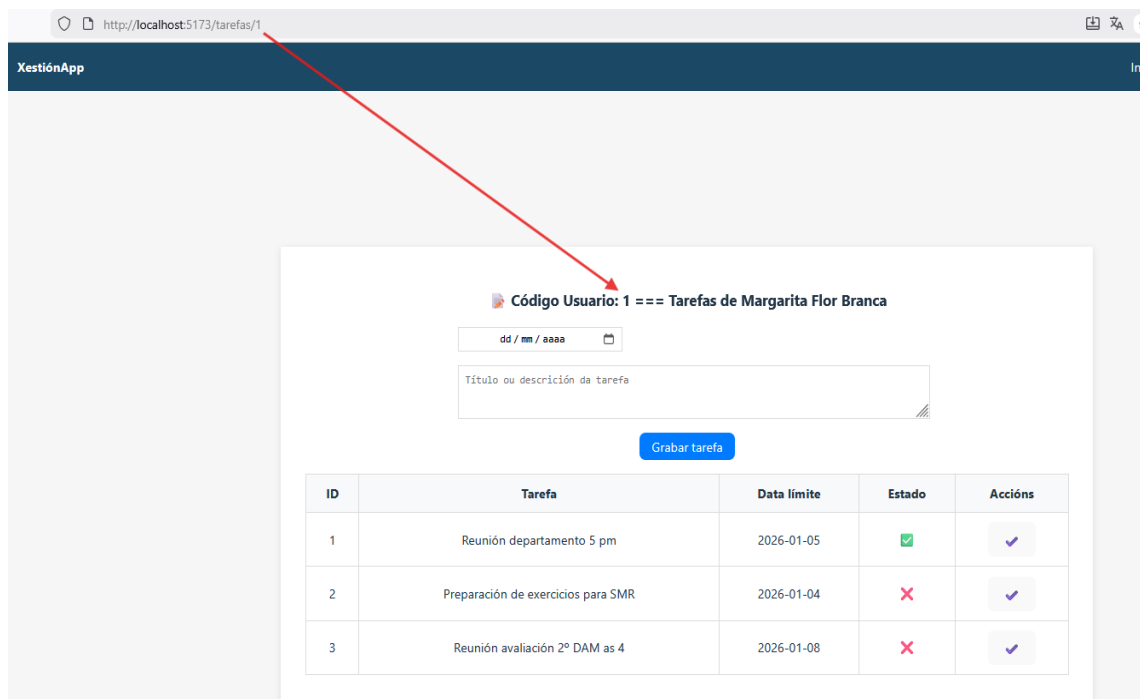
Quédanos unhas pequenas modificacións no **template** de ***XestionTarefas.vue*** que teñen que ver só co nome das variables para no confundilas coas de ***usuarioRoute***

```
<div class="xestion-tarefas">
  <!-- <h3 v-if="usuarioStore.id" -->
  <h3 v-if="usuario">
    <!-- Código Usuario: {{ usuarioStore.id }} === Tarefas de {{ usuarioStore.nome }} -->
    Código Usuario: {{ usuario.id }} === Tarefas de {{ usuario.nome }}
  </h3>

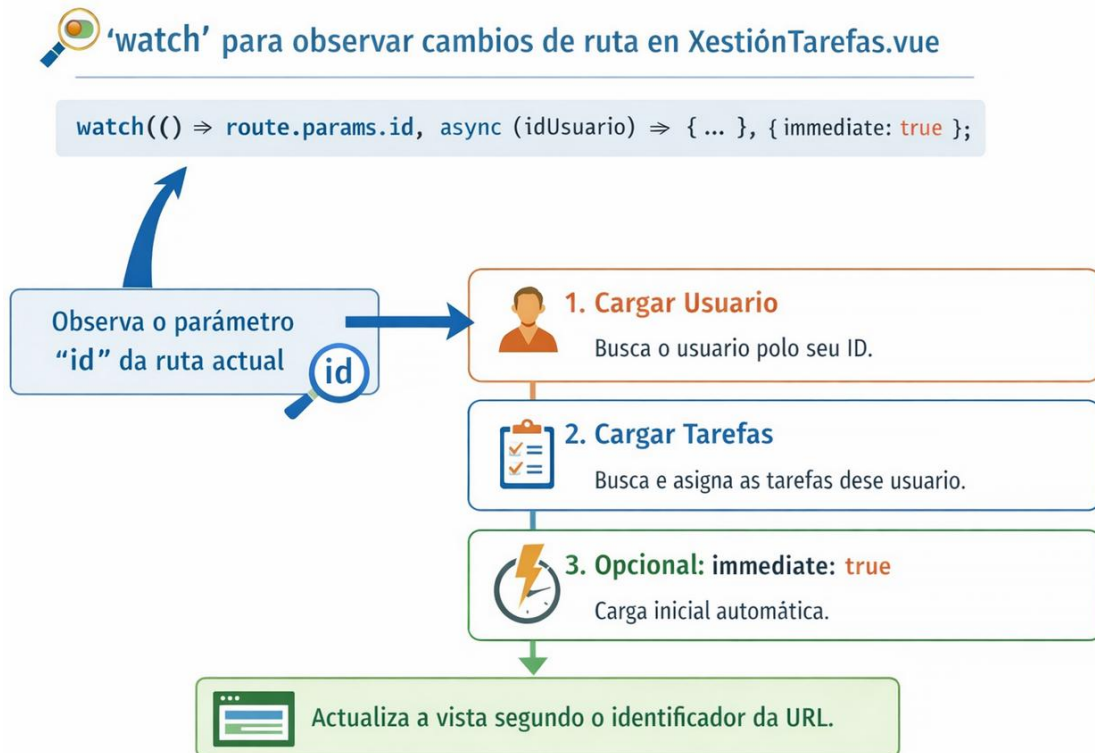
  <p v-else>Non hai usuario seleccionado</p>

  <!-- FORMULARIO NOVA TAREFA -->
  <form @submit.prevent="engadirTarefa" class="form-tarefa">
    <input v-model="novaTarefa.dataLimite" type="date" required />
```

Se nos fixamos na imaxe vemos que accedemos as tarefas de usuario polo ***id*** na **URL** ao pulsar o botón tarefas en ***XestiónUsuarios.vue*** dende onde grazas a **router-link** carga o compoñentes ***XestionTarefas.vue*** con o parámetro ***id*** do usuario.



Resumindo, usamos **watch** para reaccionar aos cambios no parámetro da ruta. Cada vez que cambia o **id** do usuario na *URL*, *Vue* recarga automaticamente o usuario e as súas tarefas. A seguinte imaxe indica a sucesión de pasos que teñen lugar.



A.6. Que escoller nunha aplicación

En Vue 3 existen distintas estratexias para compartir información entre compoñentes. Nesta unidade empregamos **dúas aproximacións complementarias**:

- o **store global (Pinia)**
- e a **reacción aos cambios da URL mediante watch**

Ambas son válidas, pero teñen **obxectivos distintos**.

Un **store** é un **almacén de estado global** accesible dende calquera compoñente da aplicación con as seguintes vantaxes:

- Código moi claro e centralizado
- Ideal para datos compartidos por moitos compoñentes
- **Moi útil para sesións, usuarios logueados, roles, preferencias**
- Evita repetir peticións *HTTP*

Pero adoece de:

- O estado **non é persistente** ao recargar a páxina
- Non queda reflectido na *URL*
- Non se pode compartir directamente cun ligazón

Con **watch** non gardamos o estado nun almacén global, senón que **delegamos esa responsabilidade na navegación**, sendo a *URL* é a *fonte da verdade* e o compoñente reacciona cando cambia o parámetro *id*

Este método ten as súas vantaxes:

- A *URL* identifica completamente a vista
- Permite recargar a páxina sen perder contexto
- Posibilidade de copiar/compartir enlaces
- Patrón habitual en aplicacións profesionais

Cal escoller? Ambas dependendo do que se necesite.

En moitas aplicacións profesionais os parámetros da URL utilízanse para identificar o recurso que se está mostrando (por exemplo */usuarios/tarefas/2*). Aquí **watch** manda

O estado global, pola súa banda, utilízase para información que debe estar dispoñible en toda a aplicación, como **sesións de usuario** ou preferencias.

Concluindo, nunha aplicación real non se escolle entre *Pinia* ou *watch*, senón que se empregan ambas ferramentas segundo o contexto e é como se traballa hoxe en día. **Pinia** utilízase para o **estado global** para ter datos dispoñibles en todo momento,

mentres que os **parámetros da ruta e *watch*** permiten construír vistas dependentes da navegación, recargables e compartibles mediante *URL*. Esta combinación dá lugar a aplicacións máis claras, escalables e profesionais.

A.7. Liñas futuras de aprendizaxe

Con esta unidade remata a nosa iniciación en Vue 3, Pinia e Router. Agora contamos coas bases necesarias para construír aplicacións reactivas e escalables, e temos unha visión clara das ferramentas que usaredes en proxectos máis complexos no futuro.

Aínda que o contido desta unidade pode resultar máis ou menos denso dependendo da experiencia previa de cada persoa, tomando esta base nun curso posterior poderían abordarse, entre outras, as seguintes liñas de traballo:

- **Autenticación e autorización:** calquera aplicación web profesional debe manexar aspectos como a xestión de usuarios, roles e xestión de sesións con *tokens*.
- **Vue Router avanzado:** uso de *navigation guards* (*beforeEach* e *beforeEnter*), *lazy loading* de vistas, rutas anidadas, rutas públicas e privadas e xestión de error máis alá do 404.
- **Estado avanzado con Pinia:** con aspectos como *stores* modulares, persistencia de estes en *localStorage* e *sessionStorage* ou o uso combinado de *computed* dentro da store coa sincronización do *backend* e *store*.
- **Comunicación co backend:** configuración de backends avanzados con MongoDB ou MySQL, paxinación, filtros, interceptores de *Axios*.
- **Calidade e mantemento:** coa introdución ao *testing*, documentación, boas prácticas coas carpetas, organización profesional do proxecto ou tipado con *Typescript*.
- **Xeración do build do proxecto:** paso previo a subida e posta en produción.

A.8. Resumo

- O emprego de **Pinia** permítenos xestionar **estado global da aplicación**, compartindo datos entre compoñentes e vistas **sen usar *props* nin *emits***.
- Os parámetros de ruta xunto con ***watch*** permiten crear **vistas reactivas á navegación**, recargables e dependentes da URL.
- Pinia e watch **non son excluíntes**: úsanse conxuntamente segundo se trate de estado da aplicación ou estado da navegación.